

大连海洋大学

毕 业 论 文（设 计）

微头条系统（联机 C/S 版）的设计与实现

学 生 姓 名：	姜益宏
----------	-----

指 导 教 师：	史鹏辉
----------	-----

专 业 名 称：	计算机科学与技术
----------	----------

所 在 学 院：	信息工程学院
----------	--------

2026 年 9 月

目 录

第一章 系统分析	2
1.1 可行性分析	2
1.2 需求概述与事件列表	3
1.3 业务流程分析	3
1.4 用例建模	5
1.5 静态结构分析	6
1.6 动态行为分析与设计类图	7
第二章 系统设计	9
2.1 体系结构设计	9
2.1.1 逻辑结构设计	9
2.1.2 物理结构设计	10
2.2 功能模块设计	10
2.3 数据库设计	11
2.3.1 数据库概要设计	11
2.3.2 数据库逻辑设计	15
2.4 接口契约设计	18
2.5 类与构件设计	19
2.6 界面设计	19
第三章 系统实现	19
3.1 开发与运行环境	19
3.2 客户端接口封装实现	19
3.3 桌面窗口与安全模型实现	20
3.4 服务端守卫与令牌实现	20
3.5 关键业务实现	21
3.6 视图实现	21
第四章 系统测试	22
4.1 测试方案	22
4.2 接口功能测试	22
4.3 联机场景验证	22
4.4 测试结论	22

第一章 系统分析

1.1 可行性分析

项目启动前完成三方面论证^[9]。技术可行性：Electron 与 NestJS 均为成熟开源框架，文档与

社区完善，两端以 HTTP 与 JSON 通信，联调简单；开发者仅需 JavaScript 一种语言即可覆盖全栈。经济可行性：无商业授权与专用硬件，服务端可运行于普通微机，客户端安装包可自由分发。操作可行性：桌面窗口符合传统软件使用习惯，登录页可配置服务器地址，普通用户只需获得服务端地址即可接入。三方面论证通过，项目可行。

1.2 需求概述与事件列表

系统的参与者为普通用户与管理员，另有一个隐含的前置角色——运维者，负责启动服务端并向用户发布地址。普通用户经由桌面客户端完成内容消费与创作：配置服务器地址、注册登录后浏览信息流（搜索、筛选、排序、分页），在详情页点赞、评论与回复，发布并维护自己的头条，管理个人资料；管理员通过管理面板进行用户查看、头条全量治理、类型与公告维护、排行统计与评论管理。表 2.1 按资源维度整理了系统必须响应的全部事件。

表 1.1 系统事件列表

资源	事件	系统响应	权限
连接	配置服务器地址	保存地址并用于后续全部请求	前置
认证	注册	完整性、一致性、唯一性校验后建户	公开
认证	登录	核验凭据并签发令牌	公开
认证	查询资料 / 修改密码	返回用户信息；旧口令校验后更新	登录
头条	列表查询	关键字、类型过滤与四口径排序分页	公开
头条	查看详情	浏览计数自增并返回点赞状态	公开
头条	发布 / 修改 / 删除	校验归属与内容；删除级联清理	本人或管理员
互动	点赞切换	按（用户，头条）防重并同步计数	登录
互动	评论 / 回复 / 删评论	长度校验；删除级联子评论	登录 / 本人或管理员
类型	增改删	查重与关联保护（有头条禁删）	管理员
公告	查询 / 增改删	关键字检索；维护并即时生效	公开 / 管理员
统计	热门与活跃排行	多口径排序分页输出	管理员

1.3 业务流程分析

普通用户的联机使用流程如图 2.1 所示：首次使用先在登录页配置服务器地址，随后提交账号信息；系统跨网络调用服务端完成认证，失败（地址错误或服务未启动）时给出明确的连接错误提示，成功则进入首页；此后在信息流中定位内容、进入详情互动、按需发布或维护自己的头条，最后退出登录并关闭客户端。

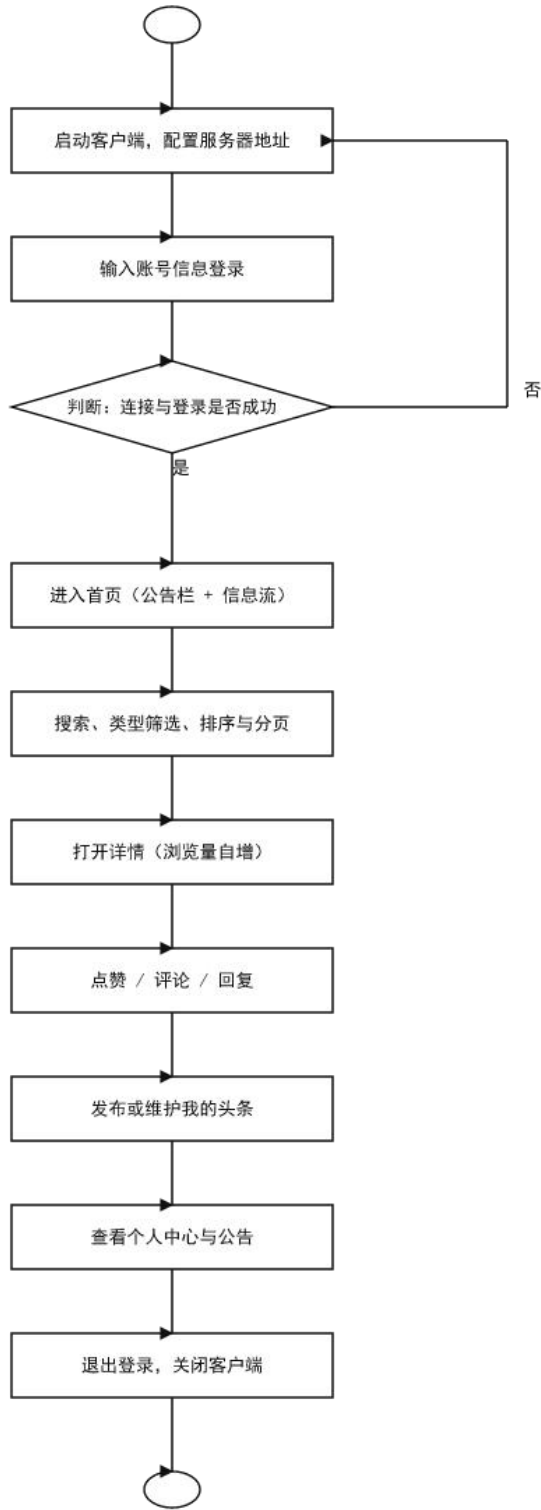


图 1.1 普通用户联机使用活动图

管理员的治理流程如图 2.2 所示：登录后进入管理面板，在用户与头条治理、类型与公告维

护、排行与评论管理三类任务间切换执行；全部治理操作经服务端守卫验证管理员身份后方可生效，完成后退出系统。

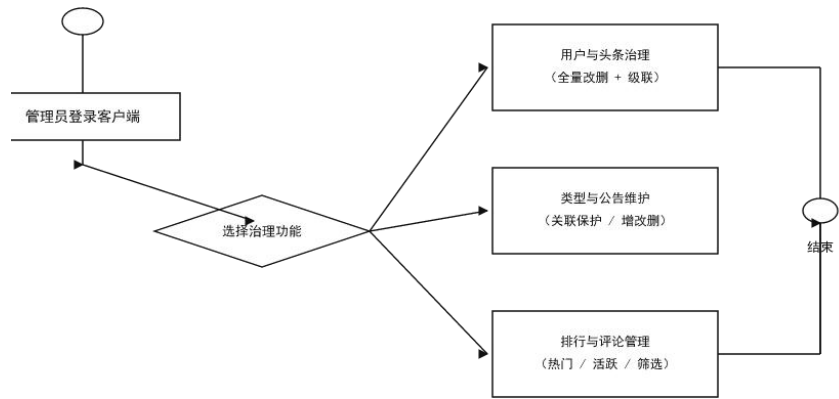


图 1.2 管理员治理活动图

1.4 用例建模

由事件列表提炼出系统用例图，如图 1.3 所示。普通用户侧包含连接配置、账户、浏览、互动、创作五组用例；管理员侧为治理用例集合，且全部管理接口受守卫保护。

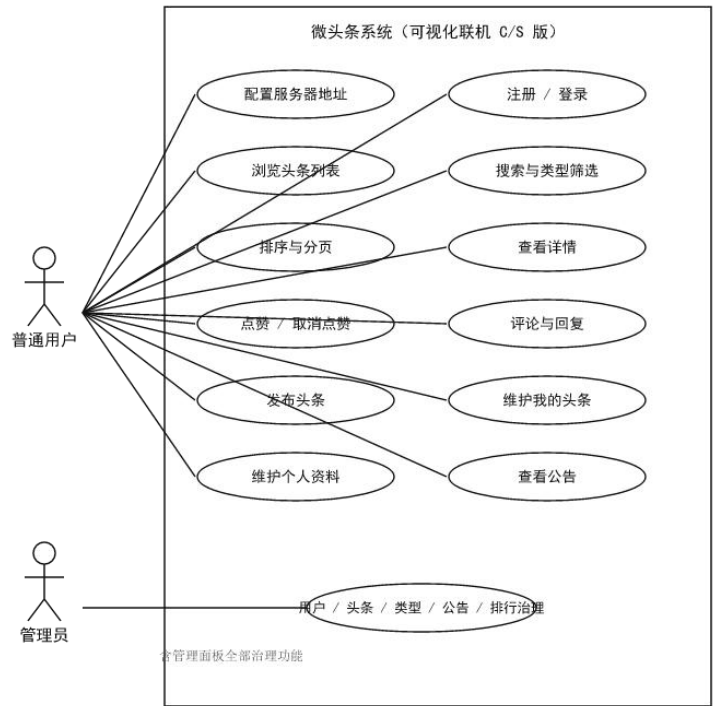


图 1.3 系统用例图

选取登录与服务器连接、删除我的头条、用户活跃排行三个用例编制描述表，见表 1.2 至表 1.4，分别覆盖联机特有流程、级联变更流程与统计查询流程。

表 1.2 用例描述：登录与服务器连接

项目	内容
用例名称	登录与服务器连接
参与者	普通用户 / 管理员
前置条件	服务端已启动；客户端已安装
主流程	1. 在登录页查看或修改服务器地址；2. 输入账号密码提交；3. 客户端跨网络调用登录接口；4. 服务端核验并签发令牌；5. 客户端保存令牌与用户信息并进入首页
备用流程	地址不可达或服务未启动时，提示无法连接服务器并允许重新配置
后置条件	令牌有效，后续请求自动携带

表 1.3 用例描述：删除我的头条

项目	内容
用例名称	删除我的头条
参与者	普通用户
前置条件	已登录；目标头条由本人发布
主流程	1. 在我的头条中定位内容；2. 触发删除并二次确认；3. 客户端调用删除接口；4. 服务端校验归属；5. 移除头条并级联清理点赞与评论；6. 列表刷新
备用流程	非本人内容（非管理员）时提示只能删除自己发布的头条
后置条件	头条及其关联数据全部清除且计数一致

表 1.4 用例描述：用户活跃排行

项目	内容
用例名称	用户活跃排行
参与者	管理员
前置条件	以管理员身份登录
主流程	1. 打开管理面板排行页；2. 请求活跃排行接口；3. 服务端按发帖数与评论数之和降序输出用户列表
备用流程	无数据时返回空列表
后置条件	纯查询，不改变数据

1.5 静态结构分析

系统的领域实体与服务端数据集合一致，共六个：用户、类型、头条、点赞、评论、公告，其属性与联系如图 2.4 所示。用户与头条、点赞、评论构成三组一对多关系，类型对头条归类，评论以父评论标识支持回复层级，头条冗余维护三项互动计数用于排序展示。该结构同时也是服务端数据文件的集合划分。

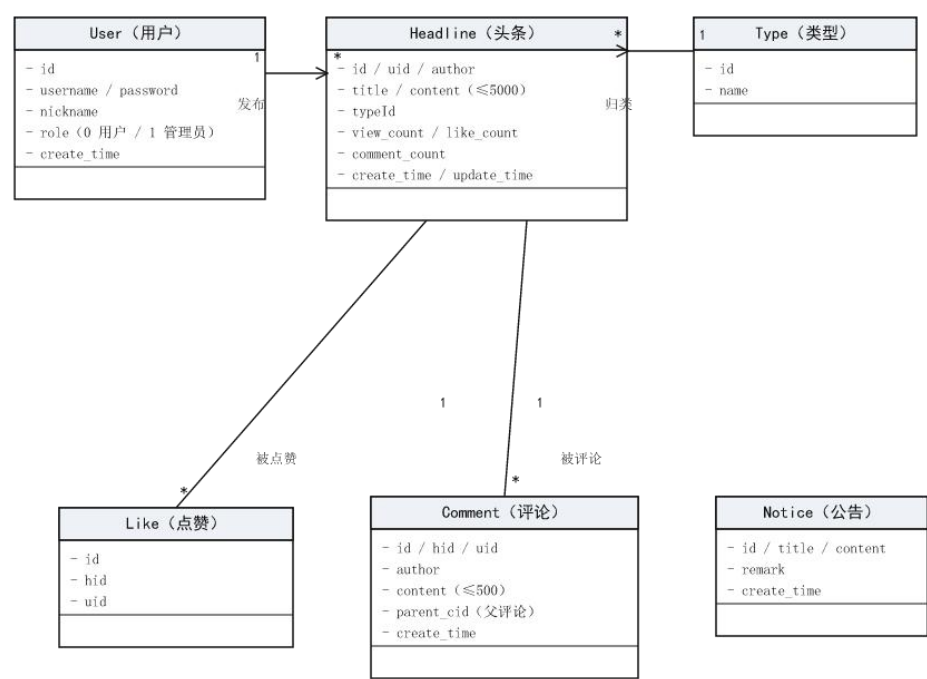


图 1.4 领域实体静态类图（仅属性）

1.6 动态行为分析与设计类图

对登录与点赞两个跨端场景建立顺序图。图 1.5 为登录场景：登录视图经接口封装向服务端发起跨网络请求，服务端核验凭据后签发令牌写入会话表并返回，客户端保存令牌进入首页；当服务器地址错误或服务未启动时，接口封装捕获网络异常并返回统一格式的失败消息。图 1.6 为点赞场景：请求先经守卫核验令牌，再由控制器在点赞集合中查重，据此新增或移除记录并同步冗余计数，落盘后返回最新状态，界面即时刷新。

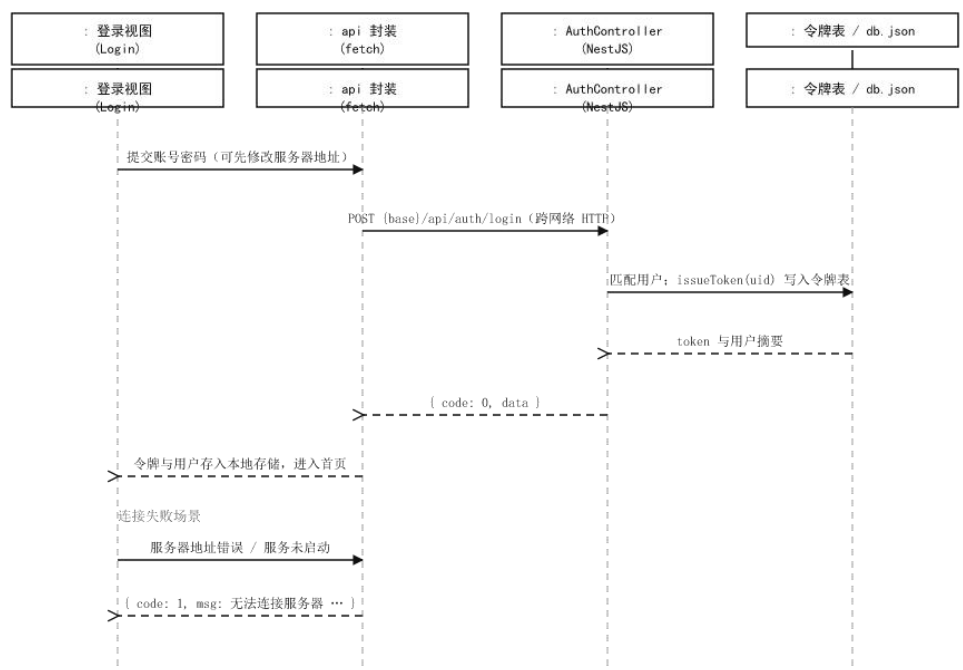


图 1.5 登录与服务器连接顺序图

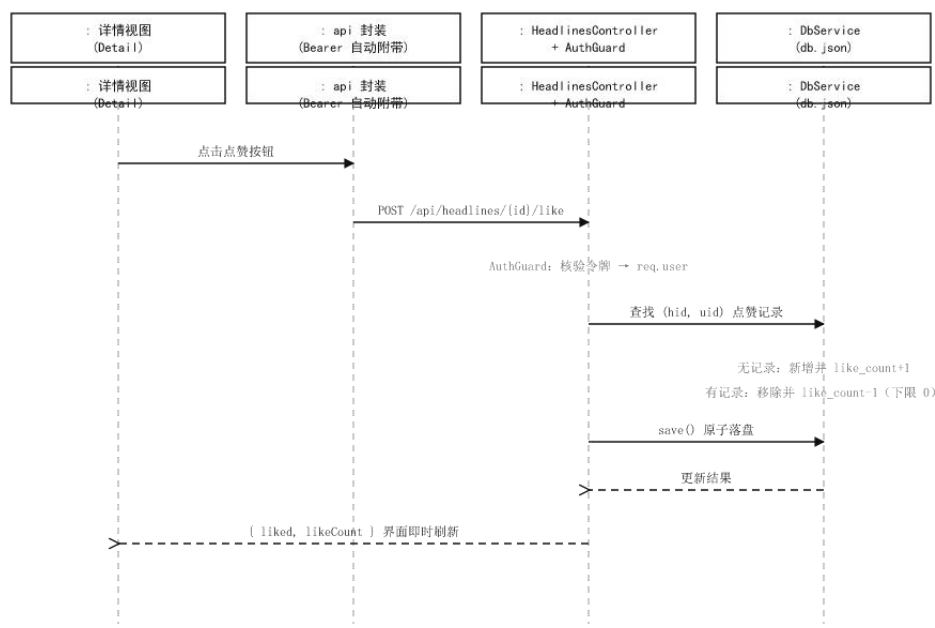


图 1.6 点赞切换顺序图

将两场景中的职责固化到结构，得到图 1.7 的设计类图：客户端分为主进程、视图与接口封装三部分，服务端分为控制器、守卫与数据服务三部分，令牌表与会话逻辑独立成可复用构件。分析与设计的衔接至此完成。

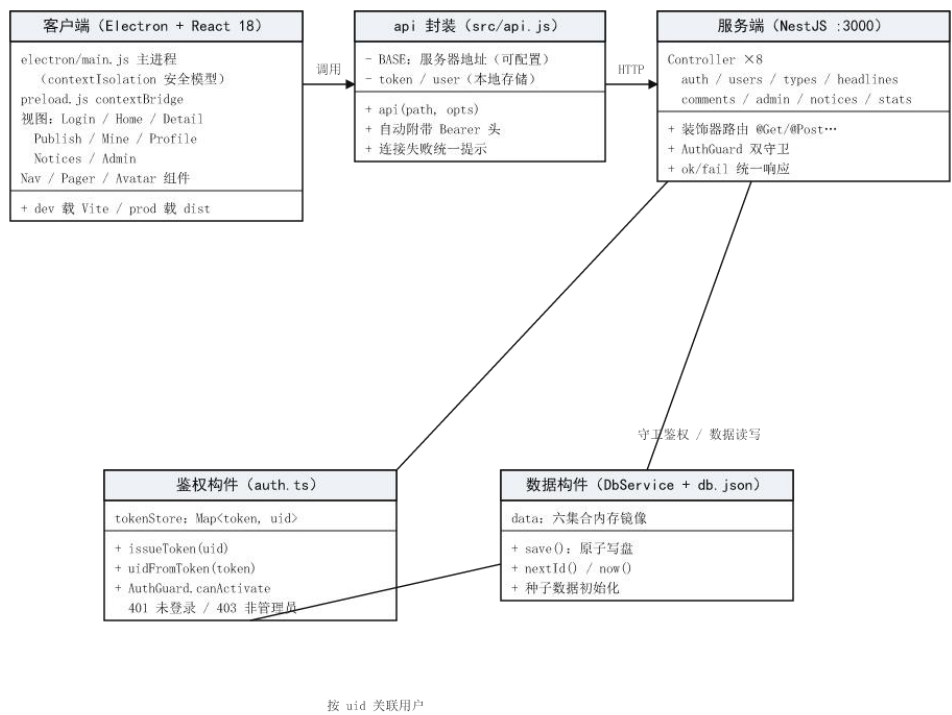


图 1.7 设计类图（属性与方法）

第二章 系统设计

2.1 体系结构设计

2.1.1 逻辑结构设计

系统在逻辑上分为客户端表示层、服务端业务层与服务端数据层，如图 3.1 所示。客户端表示层包含 Electron 主进程（以上下文隔离模式加载界面）、React 视图与统一的接口封装，接口封装集中管理服务器地址与令牌，是两端之间唯一的通道^[10]；服务端业务层为 NestJS 应用，八个控制器按资源分治，双守卫统一裁决登录与角色，响应统一包装；服务端数据层以单文件保存六个集合，采用临时文件加重命名的原子写盘策略^[11]。三层之间经跨网络 HTTP 调用与本地方法调用衔接，任一层均可独立替换。

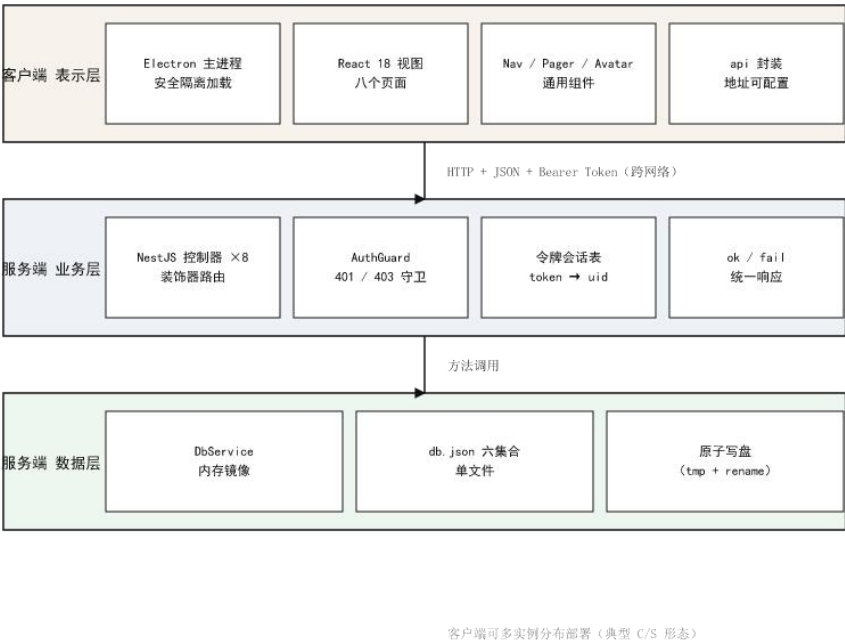


图 2.1 系统逻辑结构示意图

2.1.2 物理结构设计

物理部署如图 2.2 所示：多台用户计算机各自运行 Electron 客户端，应用服务器运行 Node 环境与 NestJS 服务端并监听 3000 端口，数据文件与服务端同机；另设模拟接口服务用于接口测试。客户端与服务端之间通过 HTTP 交换 JSON 报文并携带 Bearer 令牌，服务器地址可在客户端登录页灵活配置，构成典型的联机客户端/服务器形态。

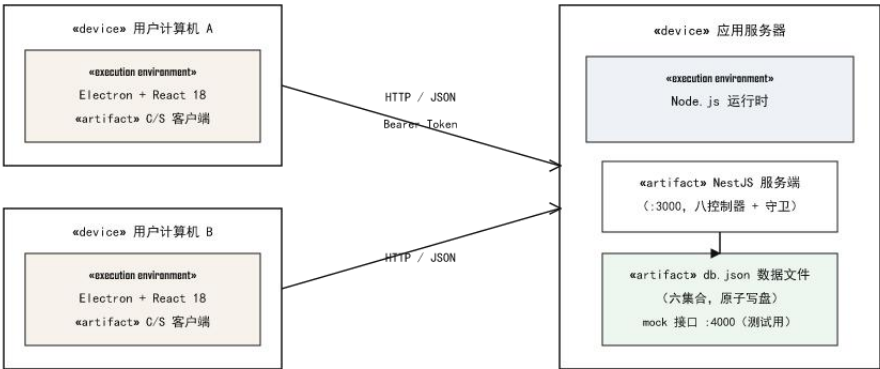


图 2.2 系统物理部署图

2.2 功能模块设计

系统功能划分为六个模块，如图 2.3 所示：连接与认证模块（含服务器地址配置）、资讯浏览模块、互动模块、创作模块、个人与公告模块、管理模块。客户端页面与服务端控制器均按模块对应组织，模块间不产生横向依赖。

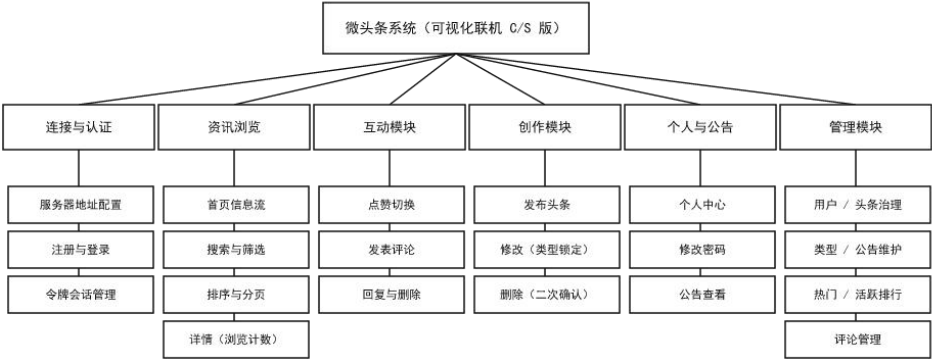


图 2.3 功能模块结构图

2.3 数据库设计

2.3.1 数据库概要设计

概念结构设计是将需求分析中识别出的业务实体及其相互关系，抽象为独立于具体数据库管理系统的概念模型。在对本系统进行数据库概念设计时，充分考虑到了系统的可扩展性和灵活性，以便在未来对系统进行修改和扩展时能够更加方便。同时，遵循了数据库设计的基本原则。本系统采用实体-联系图（E-R 图）作为概念模型的表达工具。以下逐一描述各核心实体的属性构成，最后给出系统整体的 E-R 图。

(1) 用户实体

用户实体用于存储系统注册用户的基本账号信息与权限数据。该实体的属性包括：用户 ID（全局唯一主键）、用户名（登录账号）、密码（经加密处理的密文）、昵称（显示名称）、角色（区分学生、管理员等身份）、创建时间（账号注册时间）。用户实体是系统权限控制的基础，通过角色字段实现不同身份的功能隔离

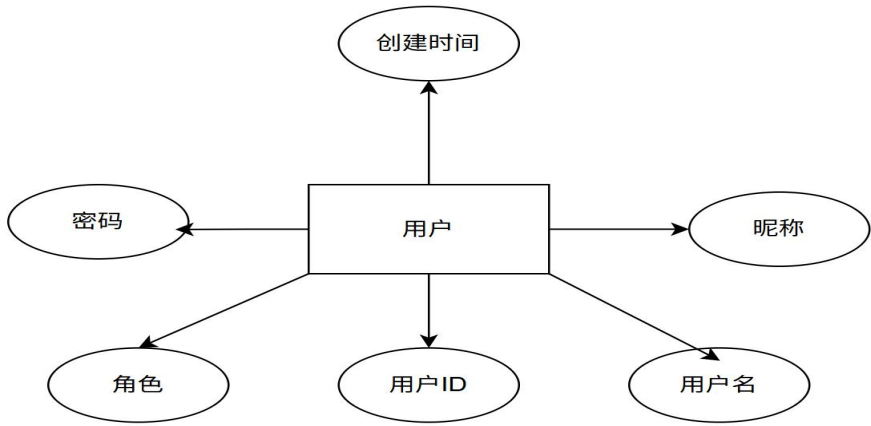


图 2.4 用户实体属性图

(2) 头条实体

头条实体用于存储系统发布的资讯、文章或公告类核心内容。该实体的属性包括：头条 ID（唯一标识）、标题（内容概要）、内容（正文详情）、类型 ID（关联分类信息）、发布者 ID（关联发布用户）、作者（署名）、浏览量（阅读次数统计）、点赞数（互动数据）、评论数（互动数据）、创建时间（首次发布时间）、更新时间（最后修改时间）。该实体是内容管理模块的核心，通过外键关联用户和类型实体，支撑内容的发布、展示与统计功能。

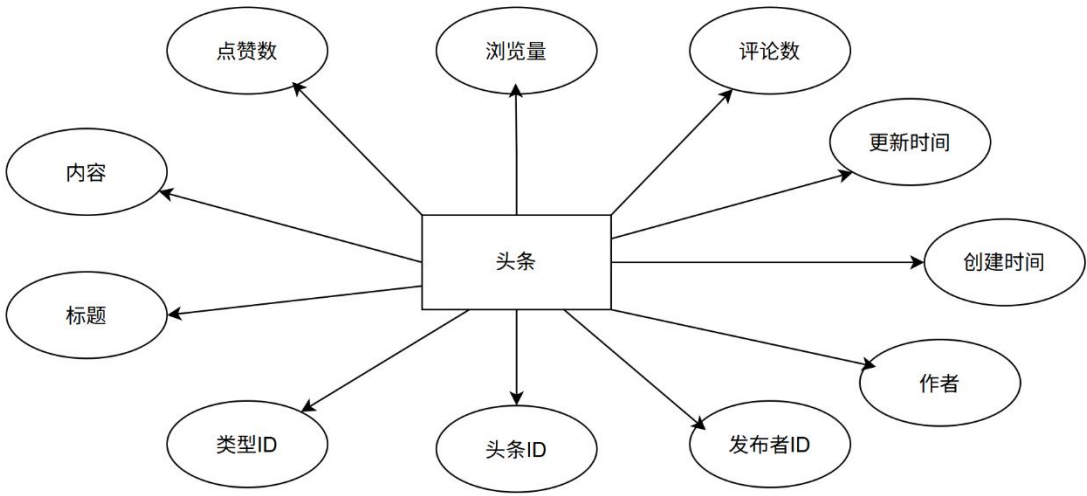


图 2.5 评论实体属性图

(3) 类型实体

类型实体用于对系统中的内容（如头条）进行归类管理，便于用户按类别浏览和检索。该实体的属性包括：类型 ID（唯一标识主键）、类型名称（分类标签，如“校园新闻”“自习室通知”等）。类型实体与头条实体之间为一对多的联系，一个类型下可以包含多条头条内容。

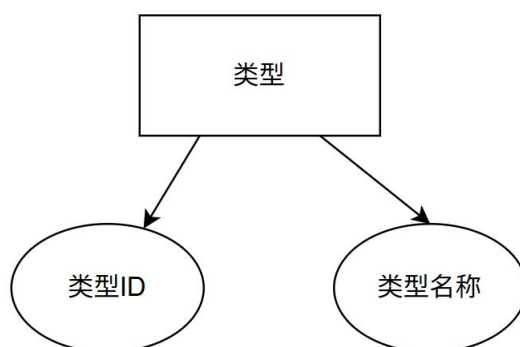


图 2.6 类型实体属性图

(4) 点赞记录实体

点赞记录实体用于存储用户对特定内容（如头条）的点赞行为，作为互动统计和用户偏好分析的数据基础。该实体的属性包括：记录 ID（唯一标识主键）、用户 ID（关联点赞用户）、头条 ID（关联被点赞的内容）。该实体通过用户 ID 和头条 ID 分别关联用户表和头条表，共同构成多对多的互动关系映射，确保同一用户对同一内容不可重复点赞。

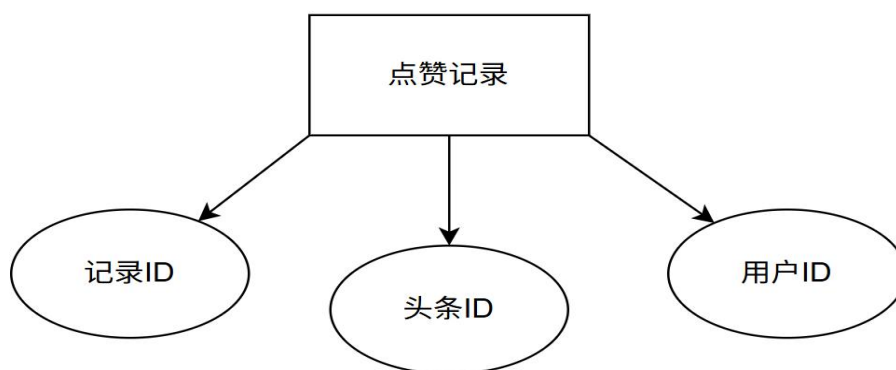


图 2.7 点赞记录实体属性图

(5) 评论实体

评论实体用于存储用户对系统内容（如头条）的评论及回复信息，支持多层级的互动交流。该实体的属性包括：评论 ID（唯一标识主键）、用户 ID（关联评论者）、头条 ID（关联被评论内容）、评论者（评论人昵称）、评论内容（具体文字）、父评论 ID（关联上级评论，用于实现回复功能）、创建时间（评论发布时间）。该实体通过用户 ID 和头条 ID 关联核心业务表，通过父评论 ID 实现评论的嵌套与追溯。

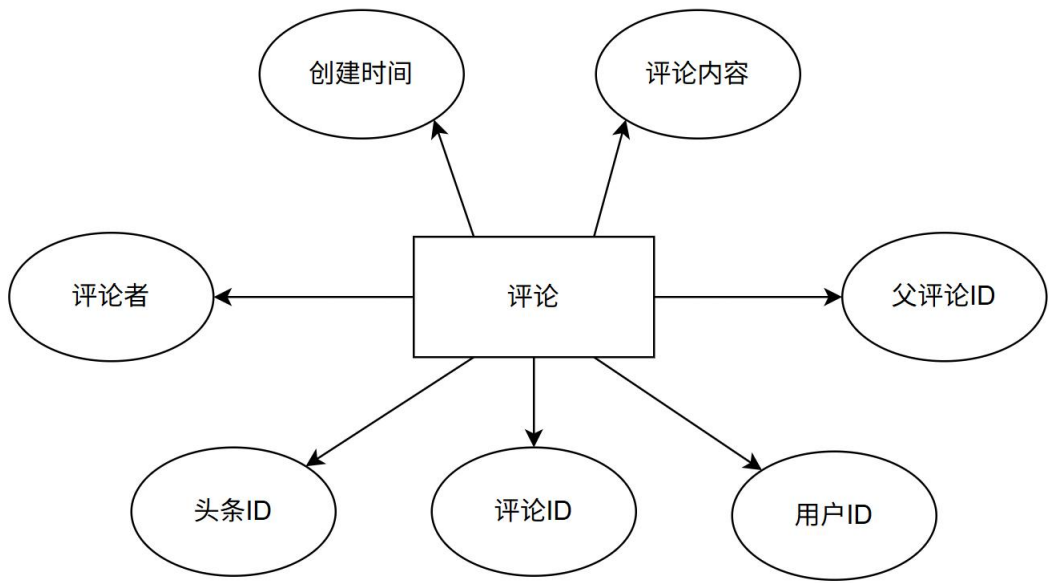


图 2.5 评论实体属性图

(6)公告实体

公告实体用于存储管理员发布的系统公告信息。该实体的属性包括：公告 ID（唯一标识主键）、标题（公告名称）、内容（公告正文）、备注（附加说明或补充信息）、创建时间（发布时间）。公告信息将展示在系统前端首页或公告栏区域，用于通知用户有关系统调整、临时闭馆、规则变更等重要信息。

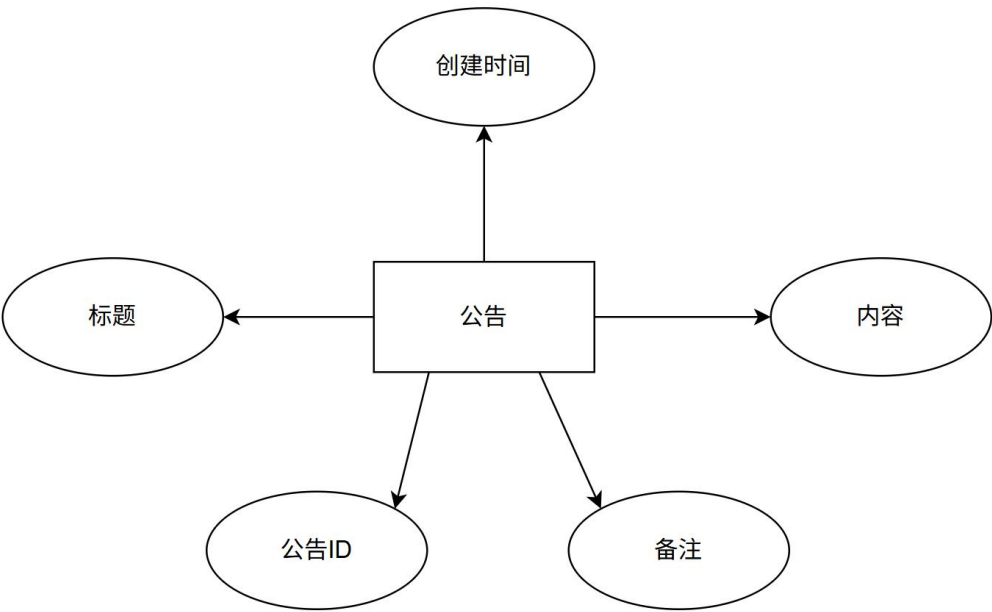


图 2.7 公告实体属性图

综合上述各实体的分析，可以绘制系统整体的 E-R 图。在整体 E-R 图中，各实体之间的关

系如下：用户实体与头条实体之间为一对多的联系，一个用户可以发布多条头条内容；用户实体与评论实体之间为一对多的联系，一个用户可以发表多条评论；用户实体与点赞记录实体之间为一对多的联系，一个用户可以进行多次点赞操作；类型实体与头条实体之间为一对多的联系，一个类型下可以包含多条头条内容；头条实体与评论实体之间为一对多的联系，一条头条内容可以关联多条评论；头条实体与点赞记录实体之间为一对多的联系，一条头条内容可以被多个用户点赞；评论实体与自身之间存在一对多的自关联联系，一条父评论可以包含多条子评论（回复），用于实现评论的嵌套与追溯；公告实体相对独立，由管理员进行发布与维护，不与其他业务实体直接关联。

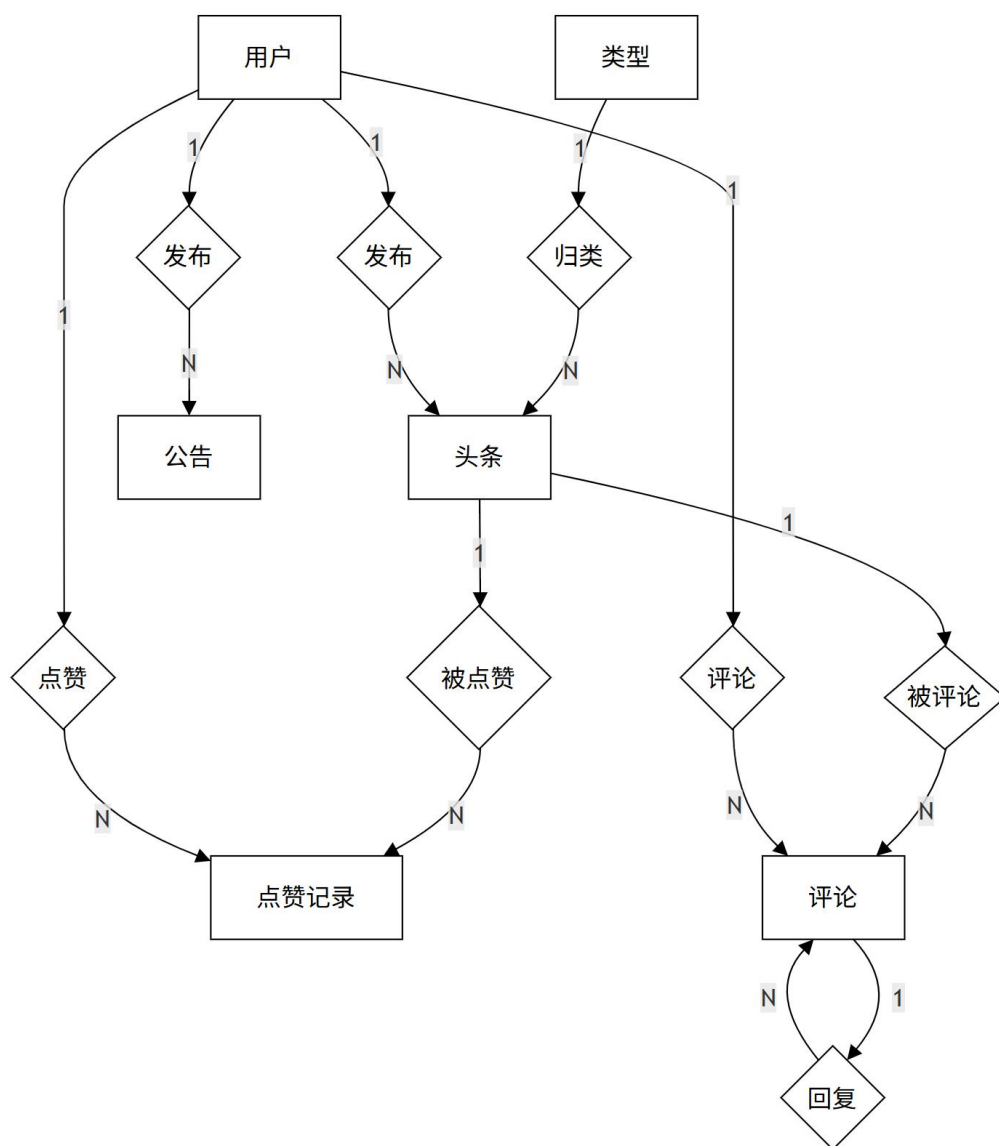


图 2.8 总体系统 er 图

2.3.2 数据库逻辑设计

本系统采用本地 db.json 文件完成数据持久化，未引入传统关系型数据库，但为清晰界定领域实体的字段结构、关联约束与业务规则，参照关系数据库范式设计六张逻辑数据表，分别对应用户、头条、类型、点赞记录、评论、公告六大实体。各表通过主键字段唯一标识记录，通过外键建立实体间的关联关系；部分展示类字段采用冗余存储设计，避免查询时频繁联表，适配单机一体化系统的查询性能需求。

(1) 用户表

存储系统全部注册用户基础信息，区分普通用户与管理员角色；用户名字段设置唯一约束，作为登录账号用于身份校验；role 字段用于控制后台管理面板的访问权限。

表 2.1 用户表

字段名称	类型	长度	主键	备注
id	bigint	20	是	用户 ID
username	varchar	50	否	用户名（登陆账号键，唯一）
password	varchar	255	否	密码
nickname	varchar	50	否	用户昵称
role	varchar	20	否	用户角色（user 普通用户 /admin 管理员）
created_at	datetime	-	否	创建时间

(2) 头条表

保存用户发布的微头条资讯主体内容；关联类型表实现内容分类，关联用户表标记发布作者；冗余存储作者昵称以省去关联查询；维护点赞、浏览、评论三项统计计数字段，用于列表排序与展示；内容修改时自动更新更新时间字段。

表 2.2 头条表

字段名称	类型	长度	主键	备注
id	bigint	20	是	头条 ID
title	varchar	255	否	标题
content	text	-	否	内容
type_id	bigint	20	否	类型 ID(外键，关联类型表)
author_id	bigint	20	否	发布者 ID（外键，关联用户表）
author_name	varchar	50	否	作者昵称（冗余存储，避免联表查询）
like_count	int	11	否	点赞数
view_count	int	11	否	浏览量

字段名称	类型	长度	主键	备注
comment_count	int	11	否	评论数
created_at	datetime	-	否	创建时间
updated_at	datetime	-	否	更新时间

（3）类型表

微头条内容分类字典表，由管理员端维护；头条记录通过 type_id 字段关联本表；当存在头条数据引用该类型记录时，禁止删除对应类型，防止产生无类型归属的孤岛数据。

表 2.3 类型表

字段名称	类型	长度	主键	备注
id	bigint	20	是	类型 ID
name	varchar	50	否	更新时间

（4）点赞记录表

记录用户对头条的点赞行为；联合 headline_id 与 user_id 字段形成唯一约束，保证同一个用户对同一条头条只能点赞一次；删除头条时级联清除本表中对应的全部点赞记录，保持数据一致性。

表 2.4 点赞记录表

字段名称	类型	长度	主键	备注
id	bigint	20	是	记录 ID
headline_id	bigint	20	否	头条 ID（外键，关联头条表）
user_id	bigint	20	否	用户 ID（外键，关联用户表）

（5）评论表

存储头条下的评论及评论回复数据；通过 parent_id 字段实现两级评论回复结构，取值为 0 代表顶层主评论，非 0 代表回复对应父评论；冗余存储评论者昵称；头条删除时级联删除全部下属评论，删除父评论时同步清理子回复，同时同步更新头条表的评论计数字段。

表 2.5 评论表

字段名称	类型	长度	主键	备注
id	bigint	20	是	评论 ID
content	varchar	1000	否	评论内容
commentator	varchar	50	否	评论者昵称（冗余存储）
user_id	bigint	20	否	用户 ID（外键，评论发布者）
headline_id	bigint	20	否	头条 ID（外键，所属头条）

字段名称	类型	长度	主键	备注
parent_id	bigint	20	否	父评论 ID, 0 表示顶级评论 (支持两级回复)
created_at	datetime	-	否	创建时间

（6）公告表

系统公告数据表，仅管理员可进行新增、修改、删除操作；首页公告栏读取本表数据进行展示；remark 字段用于管理员内部备注，不对普通用户前端展示。

表 2.6 公告表

字段名称	类型	长度	主键	备注
id	bigint	20	是	公告 ID
title	varchar	255	否	标题
content	text	-	否	内容
remark	varchar	255	否	备注
created_at	datetime	-	否	创建时间

2.4 接口契约设计

两端之间以 REST 风格接口为契约^[12]：资源名词化、方法语义化、JSON 传输、统一返回码包装，鉴权信息以 Bearer 令牌随请求头传递^[13]。接口清单见表 3.1，其中管理类接口全部要求管理员角色，普通业务接口要求有效令牌。

表 2.7 接口清单

方法	路径	语义	权限
POST	/api/auth/register	注册	公开
POST	/api/auth/login	登录换取令牌	公开
GET / PUT	/api/auth/profile 与 password	资料查询 / 密码修改	登录
GET	/api/users	用户列表	管理员
GET / POST / PUT / DELETE	/api/types	类型查询与维护	公开 / 管理员
GET / POST	/api/headlines	列表查询 / 发布	公开 / 登录
GET / PUT / DELETE	/api/headlines/:id	详情 / 修改 / 删除	公开 / 本人或管理员
POST	/api/headlines/:id/like	点赞切换	登录
GET / POST	/api/headlines/:id/comments	评论列表 / 发表	公开 / 登录
DELETE	/api/comments/:id	删除评论（级联）	本人或管理员
GET / POST / PUT / DELETE	/api/notices	公告查询与维护	公开 / 管理员

方法	路径	语义	权限
GET	/api/admin/comments 与 headlines	全量评论 / 头条治理	管理员
GET	/api/stats/hot 与 users	热门排行 / 活跃排行	管理员

2.5 类与构件设计

两端核心构件的职责分配见表 2.。设计遵循三条原则：客户端不实现任何业务规则，只负责呈现与请求；服务端不信任任何请求，鉴权与校验全部前置；数据访问收敛到独立服务，控制器不直接触碰文件。

表 2.8 核心构件职责设计

构件	端	职责要点
electron/main.js 主进程	客户端	窗口创建；开发加载 Vite、生产加载构建产物；上下文隔离
preload.js	客户端	contextBridge 注入环境信息，隔离系统能力
八个视图组件	客户端	登录、首页、详情、发布、我的、个人、公告、管理
Nav / Pager / Avatar	客户端	导航、分页、头像等通用组件
api 封装	客户端	地址配置、令牌附带、响应解析、连接失败提示
八个控制器	服务端	auth / users / types / headlines / comments / admin / notices / stats
AuthGuard 双守卫	服务端	登录守卫与管理员守卫，未过返回未授权或禁止
令牌构件	服务端	签发、解析、会话表维护
DbService	服务端	内存镜像、原子落盘、标识生成、种子数据

2.6 界面设计

客户端界面遵循桌面软件的布局习惯：顶部导航条承载搜索框、页面入口与用户身份，内容区按页面职责切换，分页与列表组件全站复用；表单校验消息就地呈现，删除类操作一律二次确认。登录页额外提供服务器地址编辑项，并以不同颜色区分连接配置区与账号输入区，强调其前置地位。全部视图以状态驱动渲染，数据变化即时反映到界面。

第三章 系统实现

3.1 开发与运行环境

环境配置为：Windows 11 操作系统；客户端基于 Electron 与 React 18，构建工具为 Vite；服务端基于 NestJS 框架运行于 Node.js 环境，监听 3000 端口；开发工具为 Visual Studio Code。启动顺序为先服务端后客户端：服务端以服务方式常驻，客户端以开发或生产模式启动窗口，登录页默认连接本机服务端并支持修改。

3.2 客户端接口封装实现

客户端全部网络交互收敛于一个封装函数：读取可配置的服务器地址，自动附带 Bearer 令牌与 JSON 请求头，捕获网络异常并转换为统一格式的失败结果，使各视图无需重复处理连接问题。实现节选如下：

```
export const getBase = () =>
  localStorage.getItem(KEY) || 'http://localhost:3000'
export const setBase = (v) =>
  localStorage.setItem(KEY, v.replace(/\/+$/, ''))

export async function api(path, opts = {}) {
  try {
    const res = await fetch(getBase() + '/api' + path, {
      ...opts,
      headers: {
        'Content-Type': 'application/json',
        Authorization: 'Bearer ' + getToken(),
        ...(opts.headers || {}),
      },
      body: opts.body ? JSON.stringify(opts.body) : undefined,
    })
    return await res.json()
  } catch {
    return { code: 1, msg: '无法连接服务器 ' + getBase() }
  }
}
```

3.3 桌面窗口与安全模型实现

Electron 主进程负责创建应用窗口：启用上下文隔离并关闭页面内 Node 能力，系统级能力经预加载脚本以受控方式注入；开发模式加载 Vite 开发服务器地址以便热更新，生产模式直接加载本地构建产物，两种模式对渲染层完全透明。实现节选如下：

```
const createWindow = () => {
  const win = new BrowserWindow({
    width: 1180, height: 800,
    title: '微头条 · C/S 客户端',
    webPreferences: {
      contextIsolation: true,
      nodeIntegration: false,
      preload: path.join(__dirname, 'preload.js'),
    },
  })
  if (process.env.VITE_DEV_SERVER_URL) {
    win.loadURL(process.env.VITE_DEV_SERVER_URL)
  } else {
    win.loadFile(path.join(__dirname, '..', 'dist', 'index.html'))
  }
}
```

3.4 服务端守卫与令牌实现

服务端鉴权由守卫统一完成：守卫从请求头解析 Bearer 令牌，经会话表还原用户；令牌无效返回未授权，管理员守卫在此基础上校验角色，非管理员返回禁止访问^[13]。令牌以随机数与时间戳合成并存储于内存会话表。守卫核心实现如下：

```
export const tokenStore = new Map<string, number>();
```

```

export function issueToken(uid: number): string {
  const token = 'tk_' + Math.random().toString(36).slice(2)
    + Date.now().toString(36);
  tokenStore.set(token, uid);
  return token;
}

@Injectable()
export class AuthGuard implements CanActivate {
  constructor(private readonly needAdmin = false) {}
  canActivate(ctx: ExecutionContext): boolean {
    const req = ctx.switchToHttp().getRequest();
    const m = (req.headers['authorization'] || '')
      .match(/^Bearer\s+(.+)$/i);
    const uid = m ? uidFromToken(m[1].trim()) : null;
    if (uid == null)
      throw new UnauthorizedException('未登录或 token 失效');
    const user = /* 按标识查回用户 */ null;
    req.user = user;
    if (this.needAdmin && user.role !== 1)
      throw new ForbiddenException('需要管理员权限');
    return true;
  }
}

```

3.5 关键业务实现

业务规则集中在控制器与数据服务中：注册执行完整性、一致性与唯一性三重校验；点赞按（用户，头条）二元组防重并同步冗余计数；删除头条与评论执行级联清理并保持计数口径一致，落盘统一采用临时文件加重命名的原子策略^[14]。以注册为例，其校验链与建户逻辑如下：

```

@Post('register')
register(@Body() b: any) {
  const { username, password, confirm, nickname } = b || {};
  if (!username || !password || !confirm || !nickname)
    throw fail('请填写完整信息');
  if (password !== confirm) throw fail('两次密码不一致');
  if (this.db.data.users.some((u) => u.username === username))
    throw fail('用户名已存在');
  const user = { id: this.db.nextId('users'), username,
    password, nickname, role: 0, create_time: this.db.now() };
  this.db.data.users.push(user);
  this.db.save();
  return ok({ id: user.id, username, nickname, role: 0 });
}

```

3.6 视图实现

客户端八个视图以状态驱动切换：登录视图负责账号输入与服务器地址配置；首页视图组合公告栏、搜索、筛选、排序与分页；详情视图承担点赞切换与评论回复；管理面板按用户、头条、类型、公告、排行、评论六区组织。导航、分页与头像为全站复用的通用组件，保证界面一致性。

第四章 系统测试

4.1 测试方案

测试以黑盒方法设计用例^[5]，分接口级与联机级两个层面：接口级以请求工具直接调用服务端，覆盖正常、异常与权限分支；联机级在两台客户端同时连接同一服务端，验证数据共享与并发表现。服务端另提供模拟接口服务用于工具压测。系统预置管理员与两个普通用户及种子数据。

4.2 接口功能测试

接口级用例共十二项，结果见表 4.1。

表 4.1 接口功能测试用例与结果

编号	验证点	操作	预期	结论
N1	正常注册	完整信息注册	建户成功并返回摘要	通过
N2	信息不全	缺少确认密码	提示请填写完整信息	通过
N3	密码不一致	两次输入不同	提示两次密码不一致	通过
N4	用户名冲突	重复注册	提示用户名已存在	通过
N5	正常登录	管理员默认账户	返回令牌与角色	通过
N6	错误口令	密码随意	提示用户名或密码错误	通过
N7	无令牌访问	省略请求头调用受保护接口	返回未登录或令牌失效	通过
N8	越权访问	普通令牌调用用户列表	返回需要管理员权限	通过
N9	点赞切换	连续两次点赞	先成功后取消，计数一致	通过
N10	级联删除	删除带互动的头条	点赞评论同步清空	通过
N11	类型保护	删除有关联的类型	拒绝并提示存在头条	通过
N12	密码修改	旧口令正确、两次一致	修改成功，旧口令失效	通过

4.3 联机场景验证

联机级验证覆盖四个场景：其一，客户端配置错误地址或服务端未启动时，登录与全部操作均返回明确的无法连接提示，修正地址后恢复正常；其二，两台客户端分别以不同账号登录，一方发布头条后另一方刷新即可见，数据共享符合预期；其三，双方同时对同一头条点赞，计数经服务端串行处理后保持准确；其四，服务端重启后业务数据完整保留，客户端因令牌失效需重新登录，行为与设计一致。

4.4 测试结论

十二项接口用例全部通过，权限双守卫、防重与级联规则均严格生效；四个联机场景表现符合

设计预期，多客户端数据一致性与网络异常处理达到目标。系统可作为联机内容社区原型稳定